DAY 16 – WIRESHARK NETWORK TRAFFIC ANALYSIS

CYBERSECURITY TRAINING

Wireshark – Network Traffic Analysis

Day 16 | Theory and Practical Learning Report

Student Name: Nethra M

Course: Cybersecurity

Assignment: Day 16

Topic: Wireshark Network Traffic Analysis

Operating System: Kali Linux

Tool: Wireshark

Virtualization Platform: Oracle VirtualBox

Assignment Type: Theory and Practical Learning

Recommended PDF filename

`Nethra_M_Day15_Wireshark.pdf`

1. Introduction

Wireshark is an open-source network protocol analyzer used to capture, inspect, and analyze network traffic in real time.

It allows cybersecurity professionals and network administrators to examine packets exchanged between devices and understand how different network protocols operate.

Wireshark provides detailed information about network communication, including source and destination addresses, protocols, packet sizes, and communication patterns.

This assignment focuses on packet capture, display filters, DNS, ICMP, TCP communication, HTTPS encryption, and the importance of network traffic analysis in cybersecurity.

2. Learning Objectives

- Understand the fundamentals of Wireshark.
- Learn the concept of packet capture.
- Identify Wireshark's main interface components.
- Understand source and destination IP addresses.
- Learn how to apply display filters.
- Analyze DNS, ICMP, and TCP traffic.
- Understand the TCP three-way handshake.
- Learn why HTTPS traffic is encrypted.
- Understand PCAP and PCAPNG file formats.
- Explore the role of Wireshark in incident response.

3. Theory – Questions and Answers

Q1. What is Wireshark?

Wireshark is a free and open-source network protocol analyzer used to capture and inspect network traffic.

It helps users examine packets, identify communication protocols, troubleshoot network problems, and investigate suspicious network activities.

Q2. What is packet capture?

Packet capture is the process of collecting network packets transmitted or received through a network interface.

It allows security professionals to examine communication details and understand how data travels between devices.

Q3. What is a capture interface?

A capture interface is a physical or virtual network interface selected in Wireshark to collect network traffic.

Examples include Ethernet, Wi-Fi, and loopback interfaces. The selected interface determines which traffic Wireshark can capture.

Q4. Name the three main Wireshark panes.

Wireshark provides three primary panes for examining captured network packets:

1. Packet List Pane: Displays captured packets with details such as packet number, time, source, destination, protocol, and length.

2. Packet Details Pane: Displays the selected packet's protocol layers and detailed information.
3. Packet Bytes Pane: Displays the raw packet data in hexadecimal and ASCII formats.

Q5. What is the difference between source and destination IP?

A source IP address identifies the device that sends a network packet, while a destination IP address identifies the device intended to receive that packet.

Both addresses are important for understanding the direction of network communication.

Q6. What does the Protocol column show?

The Protocol column in Wireshark identifies the network protocol associated with each captured packet.

Examples include TCP, UDP, DNS, HTTP, HTTPS, and ICMP.

It helps users quickly understand what type of communication is taking place.

Q7. What is a Wireshark display filter?

A Wireshark display filter is an expression used to show only packets that match specific conditions.

It helps users focus on particular protocols, IP addresses, port numbers, or communication patterns without removing the other captured packets.

Q8. Which filter shows ICMP traffic?

The following display filter shows ICMP packets:

text
```
icmp
```

It is useful for examining ping requests, replies, and other ICMP communication.

Q9. Which filter shows DNS traffic?

The following display filter shows DNS packets:

text
```
dns
```

It helps users examine DNS queries and responses used for domain name resolution.

Q10. Which filter shows TCP traffic?

The following display filter shows TCP packets:

text
```
tcp
```

It allows users to examine TCP communication, including connection establishment and data transmission.

Q11. What does `tcp.port == 443` display?

The filter `tcp.port == 443` displays TCP packets where either the source or destination port is 443.

Port 443 is commonly associated with HTTPS communication.

This filter helps users examine TCP traffic related to secure web services.

Q12. What is DNS?

DNS stands for Domain Name System.

It translates human-readable domain names into IP addresses that computers use to communicate over networks.

For example, DNS helps a browser locate a website using its domain name rather than requiring the user to remember its IP address.

Q13. What is an ICMP Echo Request?

An ICMP Echo Request is a message sent by one device to another to check network connectivity.

It is commonly generated by the `ping` command to determine whether a destination device can respond.

Q14. What is an ICMP Echo Reply?

An ICMP Echo Reply is the response sent by a destination device after receiving an ICMP Echo Request.

It indicates that the destination has responded to the connectivity test, although it does not guarantee that every network service is available.

Q15. What is the TCP three-way handshake?

The TCP three-way handshake is the process used to establish a TCP connection between two devices.

It consists of three stages:

1. SYN: The client requests a connection.
2. SYN-ACK: The server acknowledges the request and responds.
3. ACK: The client acknowledges the server's response.

After these three steps, the TCP connection is established and data communication can begin.

Q16. What are SYN, SYN-ACK and ACK?

These are TCP control flags used during connection establishment.

- SYN: Synchronizes sequence numbers and initiates a TCP connection.
- SYN-ACK: Acknowledges the SYN request and sends a synchronization request back.
- ACK: Acknowledges the received response.

Together, these flags establish a TCP connection through the three-way handshake.

Q17. Why is HTTPS content normally not visible as plain text?

HTTPS uses Transport Layer Security (TLS) to encrypt communication between a client and a server.

Encryption protects sensitive information from being easily read by unauthorized parties during transmission.

Wireshark can still display packet metadata, such as IP addresses, ports, and protocol information, but the encrypted application content is normally not readable as plain text without appropriate decryption information.

Q18. What is a PCAP/PCAPNG file?

PCAP and PCAPNG are file formats used to store captured network packets for later analysis.

- PCAP: A traditional packet capture file format.
- PCAPNG: A newer packet capture format that supports additional information, multiple interfaces, and richer packet metadata.

These files can be opened in Wireshark for offline network traffic analysis.

Q19. Why is Wireshark useful in incident response?

Wireshark is useful in incident response because it helps security professionals investigate network-related incidents by examining captured packet data.

It can help identify:

- Unusual network communication.
- Suspicious connection patterns.
- Unexpected protocols.
- DNS-related anomalies.
- Communication with unknown destinations.
- Possible signs of malicious activity.

Packet analysis helps investigators understand what happened during a network incident and supports further investigation.

Q20. Why must packet capture be performed only on authorized traffic?

Packet capture must only be performed on authorized traffic because network packets may contain private information, confidential data, and sensitive communication details.

Capturing traffic without permission can violate privacy, organizational policies, and applicable laws.

Authorized packet capture ensures that network analysis is conducted legally, ethically, and responsibly.

4. Important Wireshark Display Filters

The following table contains commonly used Wireshark display filters.

Display Filter	Purpose
`icmp`	Displays ICMP packets
`dns`	Displays DNS packets
`tcp`	Displays TCP packets
`udp`	Displays UDP packets
`http`	Displays HTTP packets
`tls`	Displays TLS packets
`tcp.port == 443`	Displays TCP packets involving port 443
`ip.addr == 127.0.0.1`	Displays packets involving the specified IP
`ip.src == 192.168.1.10`	Displays packets from a specified source IP
`ip.dst == 192.168.1.10`	Displays packets sent to a specified destination IP

Note: These are display filters, not capture filters. The IP addresses shown are examples.

5. TCP Three-Way Handshake – Communication Flow

TCP connection establishment

Client

Initiates connection

SYN

Server

Acknowledges and responds

SYN-ACK

Client

Confirms connection

ACK
Connection established

Data communication begins

6. Practical Implementation – Wireshark

The following activities can be performed in a Kali Linux laboratory environment to understand packet capture and network traffic analysis.

Activity	Purpose
Open Wireshark	Access the packet analysis interface
Select a capture interface	Choose the network interface
Start packet capture	Collect network packets
Apply `icmp`	Examine ICMP communication
Apply `dns`	Examine DNS queries and responses
Apply `tcp`	Examine TCP traffic
Apply `tcp.port == 443`	Examine TCP traffic involving HTTPS port 443
Stop capture	Finish collecting packets
Save as PCAPNG	Store captured traffic for later analysis

Example display filters

text
```
icmp
```
text
```
dns
```
text
```
tcp
```
text
```
tcp.port == 443
```

Practical note: Capture only traffic from your own device or an explicitly authorized laboratory network. HTTPS payloads will generally remain encrypted.

7. Practical Evidence – Screenshots

Include genuine screenshots from your Wireshark application to demonstrate your practical work.

Screenshot 1 – Wireshark Main Interface

Filename: `wireshark-interface.png`

Screenshot 2 – ICMP Packet Analysis

Filename: `icmp-analysis.png`

Screenshot 3 – DNS Traffic Analysis

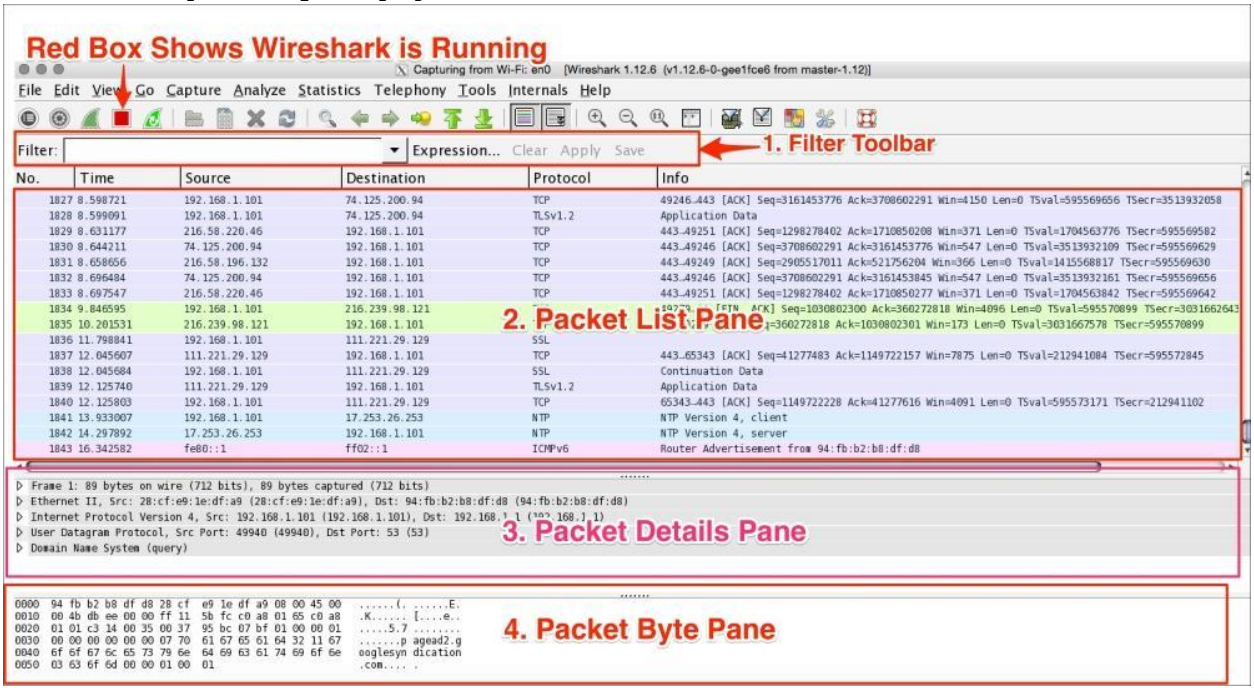
Filename: `dns-analysis.png`

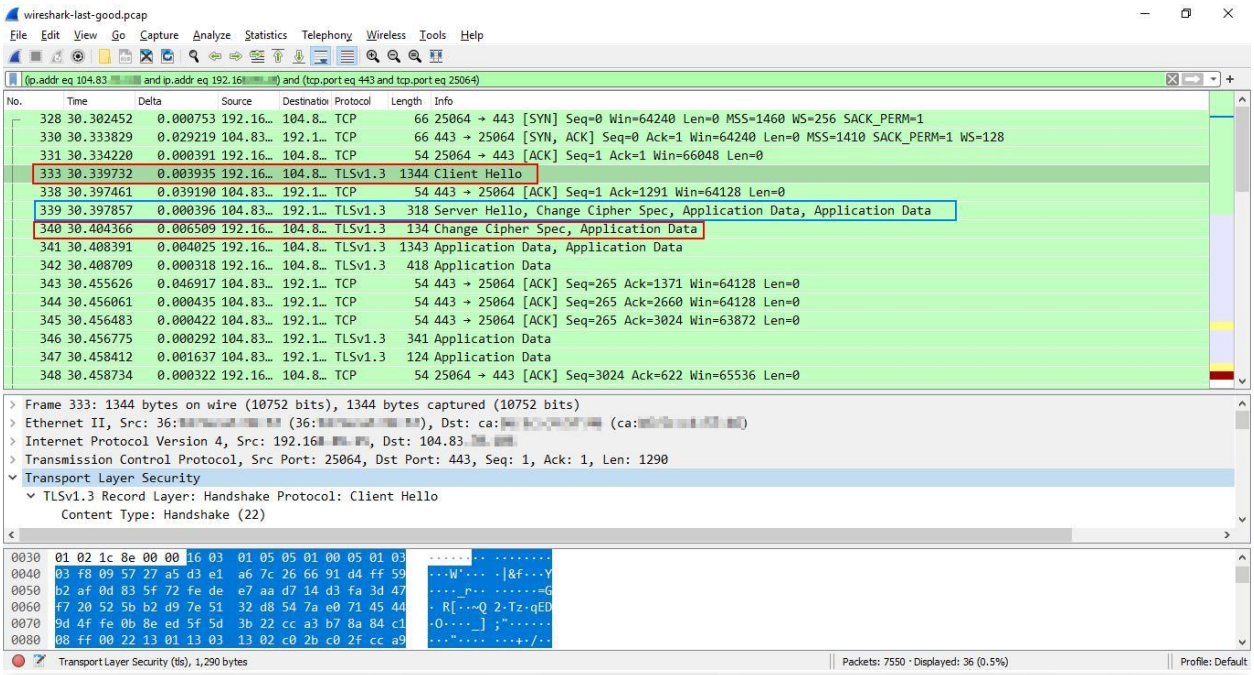
Screenshot 4 – TCP Three-Way Handshake

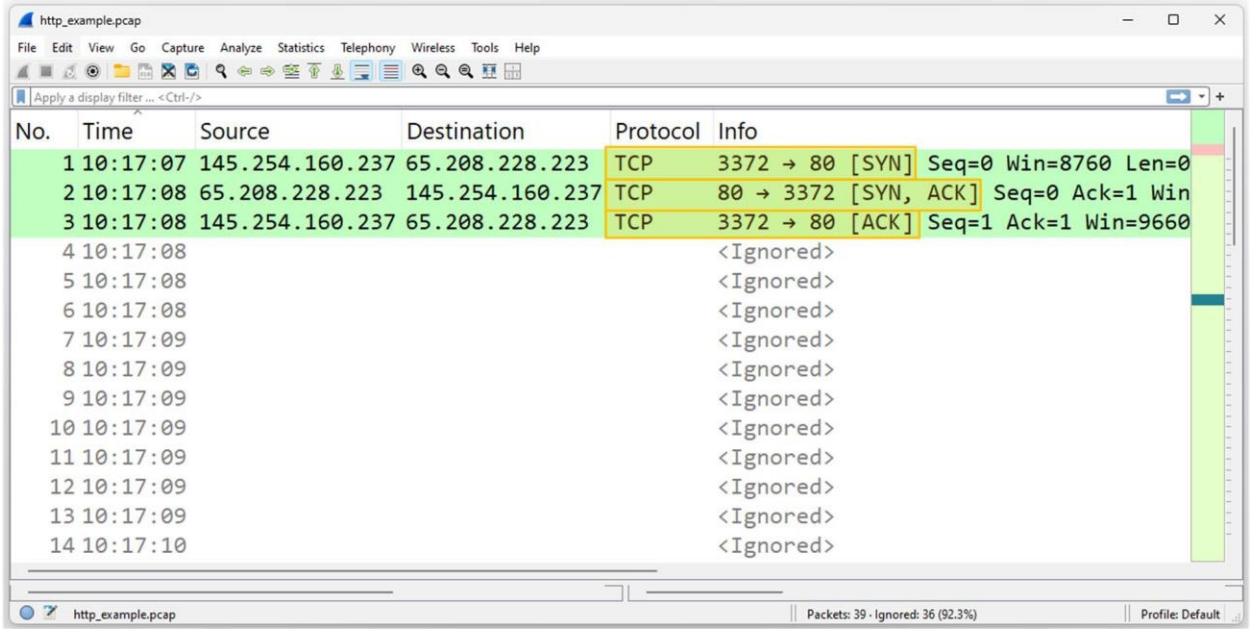
Filename: `tcp-handshake.png`

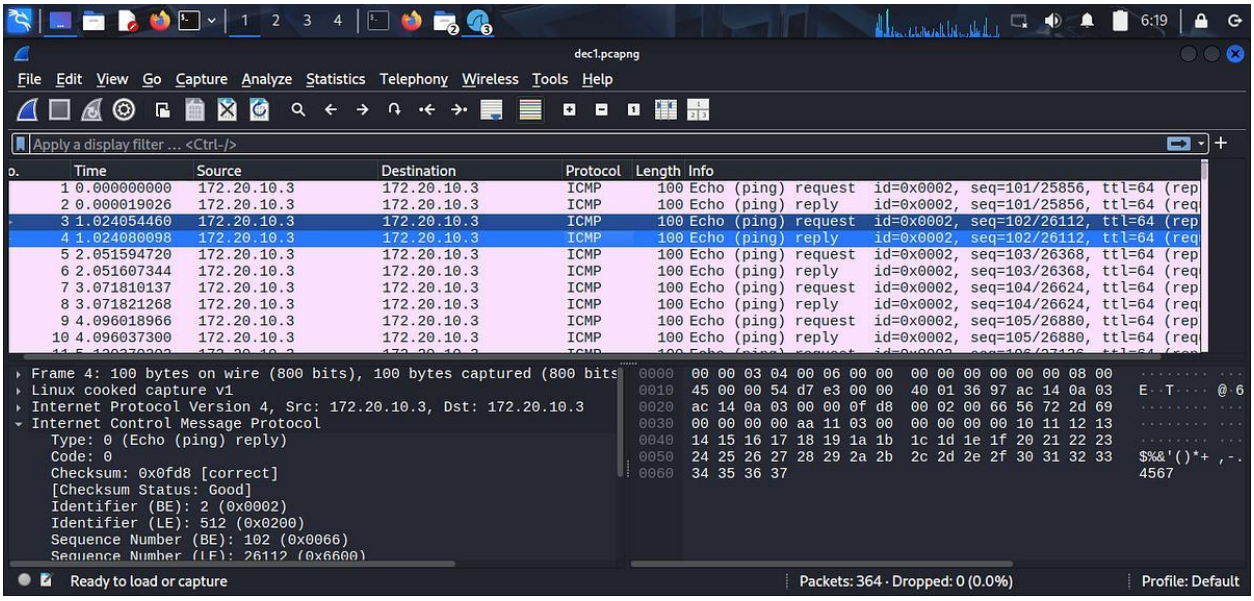
Screenshot 5 – HTTPS Traffic Analysis

Filename: `https-analysis.png`




wireshark-last-good.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
(ip.addr eq 104.83 and ip.addr eq 192.16 and (tcp.port eq 443 and tcp.port eq 25064)
No. Time Delta Source Destinatio Protocol Length Info
328 30.302452 0.000753 192.16… 104.8… TCP 66 25064 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM=1
330 30.333829 0.029219 104.83… 192.1… TCP 66 443 → 25064 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1410 SACK_PERM=1 WS=128
331 30.334220 0.000391 192.16… 104.8… TCP 54 25064 → 443 [ACK] Seq=1 Ack=1 Win=66048 Len=0
333 30.339732 0.003935 192.16… 104.8… TLSv1.3 1344 Client Hello
338 30.397461 0.039190 104.83… 192.1… TCP 54 443 → 25064 [ACK] Seq=1 Ack=1291 Win=64128 Len=0
339 30.397857 0.000396 104.83… 192.1… TLSv1.3 318 Server Hello, Change Cipher Spec, Application Data, Application Data
340 30.404366 0.006509 192.16… 104.8… TLSv1.3 134 Change Cipher Spec, Application Data
341 30.408391 0.004025 192.16… 104.8… TLSv1.3 1343 Application Data, Application Data
342 30.408709 0.000318 192.16… 104.8… TLSv1.3 418 Application Data
343 30.455626 0.046917 104.83… 192.1… TCP 54 443 → 25064 [ACK] Seq=265 Ack=1371 Win=64128 Len=0
344 30.456061 0.000435 104.83… 192.1… TCP 54 443 → 25064 [ACK] Seq=265 Ack=2660 Win=64128 Len=0
345 30.456483 0.000422 104.83… 192.1… TCP 54 443 → 25064 [ACK] Seq=265 Ack=3024 Win=63872 Len=0
346 30.456775 0.000292 104.83… 192.1… TLSv1.3 341 Application Data
347 30.458412 0.001637 104.83… 192.1… TLSv1.3 124 Application Data
348 30.458734 0.000322 192.16… 104.8… TCP 54 25064 → 443 [ACK] Seq=3024 Ack=622 Win=65536 Len=0
Frame 333: 1344 bytes on wire (10752 bits), 1344 bytes captured (10752 bits)
Ethernet II, Src: 36: (36:), Dst: ca: (ca:)
Internet Protocol Version 4, Src: 192.16, Dst: 104.83
Transmission Control Protocol, Src Port: 25064, Dst Port: 443, Seq: 1, Ack: 1, Len: 1290
Transport Layer Security
TLSv1.3 Record Layer: Handshake Protocol: Client Hello
Content Type: Handshake (22)
0030 01 02 1c 8e 00 00 16 03 01 05 05 01 00 05 01 03
0040 03 f8 09 57 27 a5 d3 e1 a6 7c 26 66 91 d4 ff 59
0050 b2 af 0d 83 5f 72 fe de e7 aa d7 14 d3 fa 3d 47
0060 f7 20 52 5b b2 d9 7e 51 32 d8 54 7a e0 71 45 44
0070 9d 4f fe 0b 8e ed 5f 5d 3b 22 cc a3 b7 8a 84 c1
0080 08 ff 00 22 13 01 13 03 13 02 c0 2b c0 2f cc a9
Transport Layer Security (tls), 1,290 bytes
Packets: 7550 · Displayed: 36 (0.5%)
Profile: Default


http_example.pcap
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Info
1 10:17:07 145.254.160.237 65.208.228.223 TCP 3372 → 80 [SYN] Seq=0 Win=8760 Len=0
2 10:17:08 65.208.228.223 145.254.160.237 TCP 80 → 3372 [SYN, ACK] Seq=0 Ack=1 Win
3 10:17:08 145.254.160.237 65.208.228.223 TCP 3372 → 80 [ACK] Seq=1 Ack=1 Win=9660
4 10:17:08 <Ignored>
5 10:17:08 <Ignored>
6 10:17:08 <Ignored>
7 10:17:09 <Ignored>
8 10:17:09 <Ignored>
9 10:17:09 <Ignored>
10 10:17:09 <Ignored>
11 10:17:09 <Ignored>
12 10:17:09 <Ignored>
13 10:17:09 <Ignored>
14 10:17:10 <Ignored>
http_example.pcap
Packets: 39 · Ignored: 36 (92.3%)
Profile: Default


dec1.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
Apply a display filter ... <Ctrl-/>
No. Time Source Destination Protocol Length Info
1 0.000000000 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) request id=0x0002, seq=101/25856, ttl=64 (rep
2 0.000019026 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) reply id=0x0002, seq=101/25856, ttl=64 (req
3 1.024054460 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) request id=0x0002, seq=102/26112, ttl=64 (rep
4 1.024080098 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) reply id=0x0002, seq=102/26112, ttl=64 (req
5 2.051594720 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) request id=0x0002, seq=103/26368, ttl=64 (rep
6 2.051607344 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) reply id=0x0002, seq=103/26368, ttl=64 (req
7 3.071810137 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) request id=0x0002, seq=104/26624, ttl=64 (rep
8 3.071821268 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) reply id=0x0002, seq=104/26624, ttl=64 (req
9 4.096018966 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) request id=0x0002, seq=105/26880, ttl=64 (rep
10 4.096037300 172.20.10.3 172.20.10.3 ICMP 100 Echo (ping) reply id=0x0002, seq=105/26880, ttl=64 (req
Frame 4: 100 bytes on wire (800 bits), 100 bytes captured (800 bits
Linux cooked capture v1
Internet Protocol Version 4, Src: 172.20.10.3, Dst: 172.20.10.3
Internet Control Message Protocol
Type: 0 (Echo (ping) reply)
Code: 0
Checksum: 0x0fd8 [correct]
[Checksum Status: Good]
Identifier (BE): 2 (0x0002)
Identifier (LE): 512 (0x0200)
Sequence Number (BE): 102 (0x0066)
Sequence Number (LE): 26112 (0x6600)
0000 00 00 03 04 00 06 00 00 00 00 00 00 00 00 08 00
0010 45 00 00 54 d7 e3 00 00 40 01 36 97 ac 14 0a 03 E··T····@·6
0020 ac 14 0a 03 00 00 0f d8 00 02 00 66 56 72 2d 69
0030 00 00 00 00 aa 11 03 00 00 00 00 00 10 11 12 13
0040 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f 20 21 22 23
0050 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f 30 31 32 33 $%&'()*+ ,-./0123
0060 34 35 36 37 4567
Ready to load or capture
Packets: 364 · Dropped: 0 (0.0%)
Profile: Default